NORAIZ RANA

Task #12

Topic

React life cycle methods and hooks useState, useEffect

Date:

22 July 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

📓 Learning Report: React Lifecycle Methods and Hooks (`useState`, `useEffect`)

1. Introduction

In React, components go through different **lifecycle phases**: **mounting**, **updating**, and **unmounting**. In class-based components, React provides **lifecycle methods** like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` to manage side effects during these phases.

With the introduction of **React Hooks** in version 16.8, we can achieve the same behavior in **functional components** using hooks like `useState` and `useEffect`.

---

2. React Lifecycle Phases (Class Components)

Phase	Method	Purpose
Mounting	`componentDidMount()`	Runs after the component is rendered
Updating	`componentDidUpdate()`	Runs after component re-renders
Unmounting	`componentWillUnmount()`	Runs before component is removed

Example:

```
class Example extends React.Component {
  componentDidMount() {
    console.log("Component mounted");
```

```
  }

  componentDidUpdate() {
    console.log("Component updated");
  }

  componentWillUnmount() {
    console.log("Component will unmount");
  }
}
```

However, in modern React development, **functional components with hooks** are preferred.

---

3. Introduction to React Hooks

Hooks allow functional components to manage state and lifecycle behaviors without writing a class.

---

4. `useState` – State in Functional Components

`useState` is a React hook that allows you to add **local component state**.

Example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
```

```
      <p>Count: {count}</p>
      <button onClick={() => setCount(count +
1)}>Increase</button>
    </>
  );
}
```

Key Points:

- `useState()` returns a state variable and a function to update it.
- Re-rendering occurs when state is updated.

---

5. `useEffect` – Managing Side Effects

`useEffect` is used for **side effects** like:

- Fetching data
- Updating the DOM
- Setting up subscriptions
- Cleaning up on unmount

Syntax:

```
useEffect(() => {
  // side effect code here
  return () => {
    // cleanup code here (optional)
  };
}, [dependencies]);
```

Example:

```
useEffect(() => {
  console.log("Component mounted or updated");
```

```
  return () => console.log("Component will
unmount");
}, [count]);
```

- The second argument (dependency array) controls when the effect runs.
- If it's empty (`[]`), it only runs on mount.
- If it includes variables, it runs whenever those change.